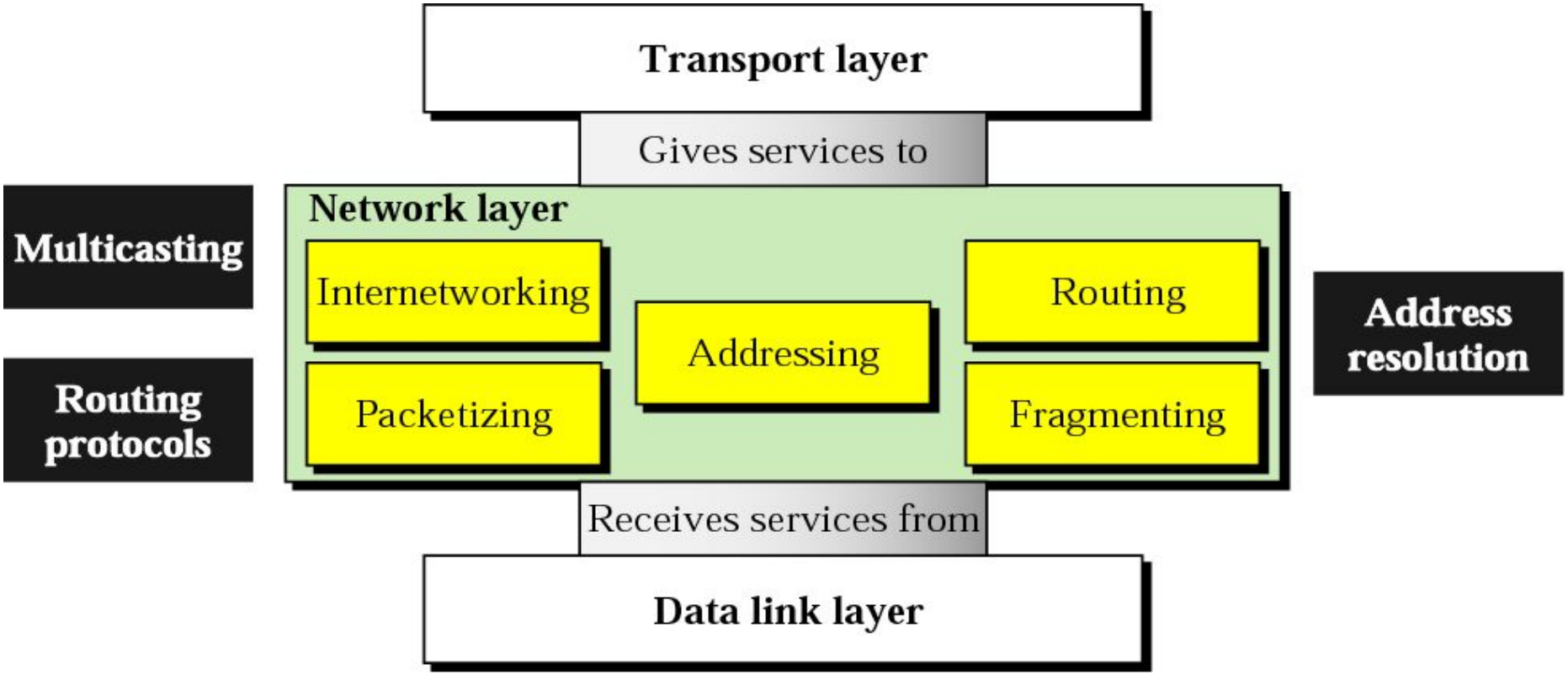
Computer Communication & Networks

Lecture 24-25

Network Layer: Delivery, Forwarding, Routing

Network Layer



Network Layer Topics to Cover

Logical Addressing

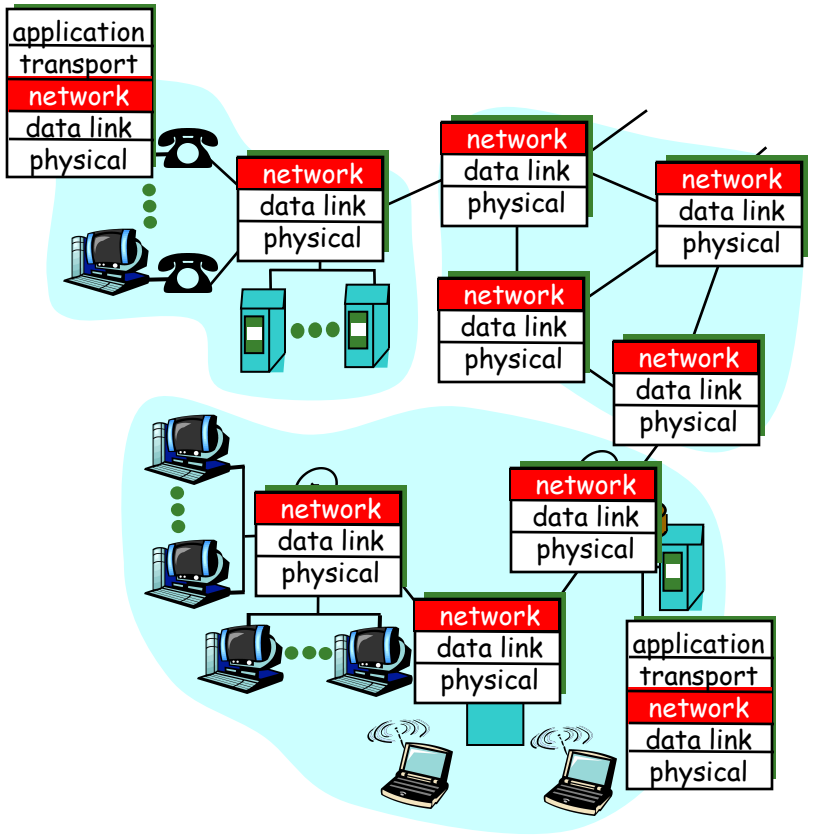
Internet Protocol

Address Mapping

Delivery, Forwarding, Routing

Network layer functions - 1

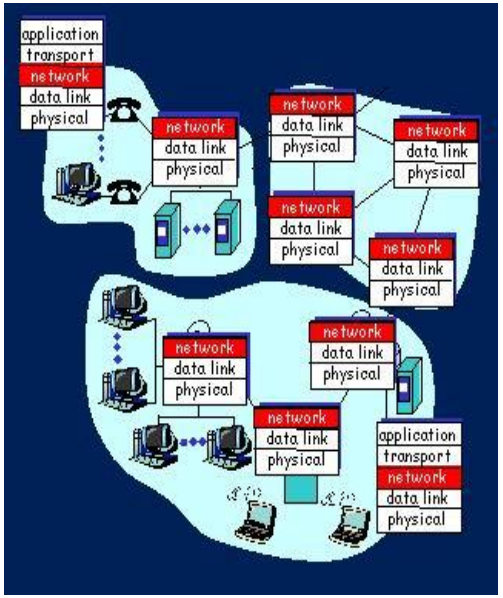
- transport packet from sending to receiving hosts
- network layer protocols in *every* host, router



Network layer functions - 2

three important functions:

- *path determination:* route taken by packets from source to dest. *Routing algorithms*
- *Switching (forwarding):* move packets from router's input to appropriate router output
- *call setup: (optional)* some network architectures require router call setup along path before data flows

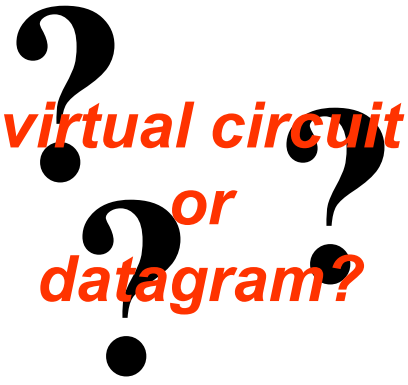


Network service model

Q: What *service model* for "channel" transporting packets from sender to receiver?

- guaranteed bandwidth?
- preservation of inter-packet timing (no jitter)?
- loss-free delivery?
- in-order delivery?
- congestion feedback to sender?

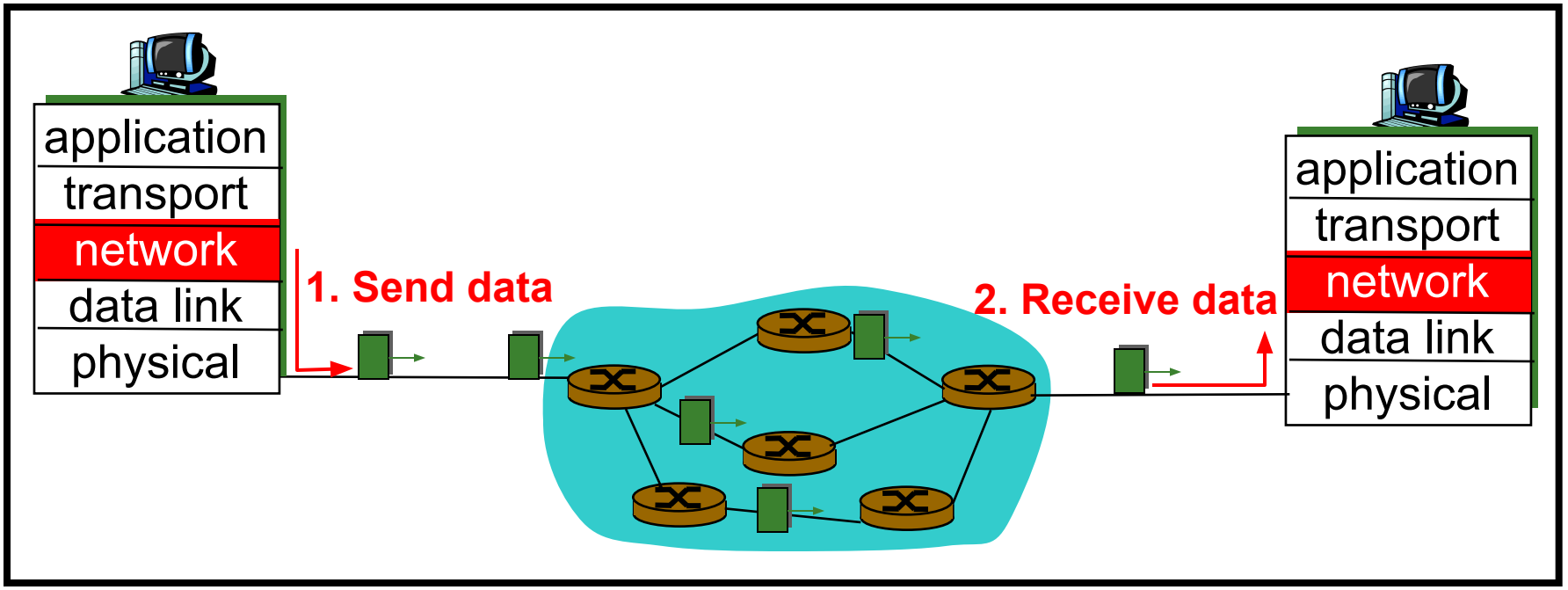
The most important abstraction provided by network layer:



Datagram networks: the Internet model - 1

- no call setup at network layer
- routers: no state about end-to-end connections
  - no network-level concept of “connection”
- packets typically routed using destination host ID
  - packets between same source-dest pair may take different paths

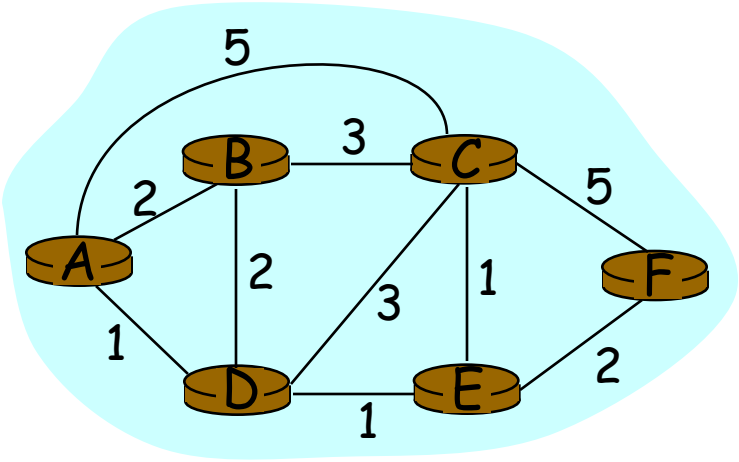
Datagram networks: the Internet model - 2



Routing

Routing protocol

Goal: determine "good" path (sequence of routers) thru network from source to dest.



- Graph abstraction for routing algorithms:
- graph nodes are routers
- graph edges are physical links
  - link cost: delay, $ cost, or congestion level

"good" path:
typically means minimum cost path
other def's possible

Routing Algorithm classification - 1

Global:

- all routers have complete topology, link cost info
- "link state" algorithms

Decentralized:

- router knows physically-connected neighbors, link costs to neighbors
- iterative process of computation, exchange of partial info with neighbors
- "distance vector" algorithms

Routing Algorithm classification - 2

Static:

- routes change slowly over time

Dynamic:

- routes change more quickly
  - periodic update
  - in response to link cost changes

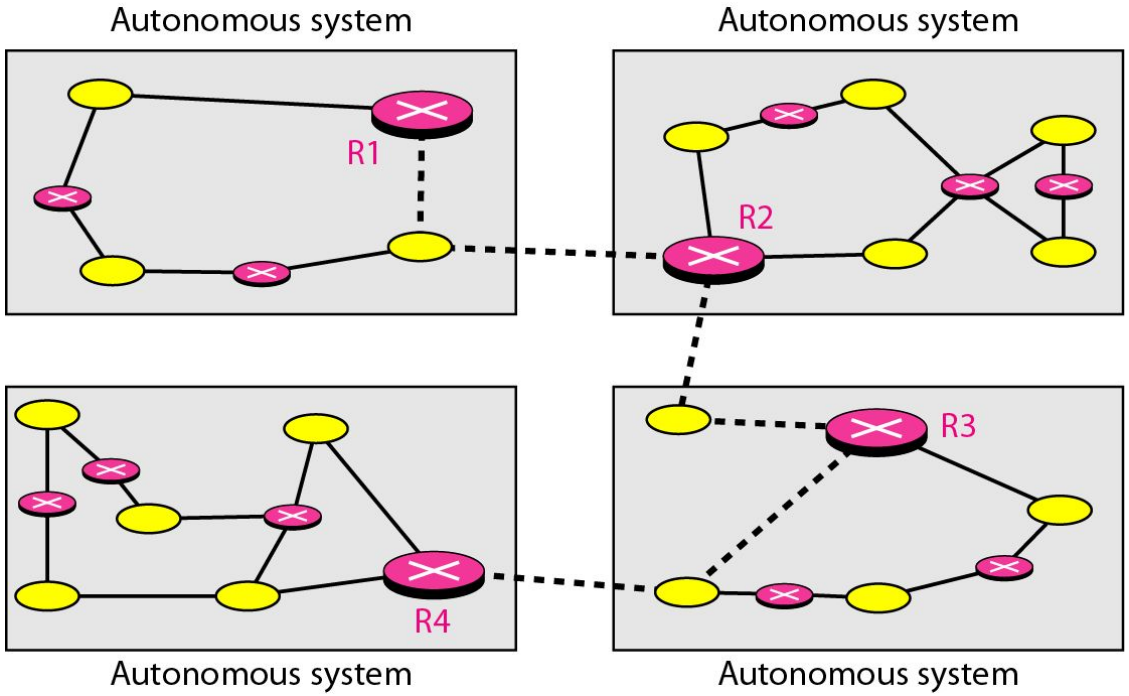
Unicast Routing Protocols

Unicast Routing Protocols

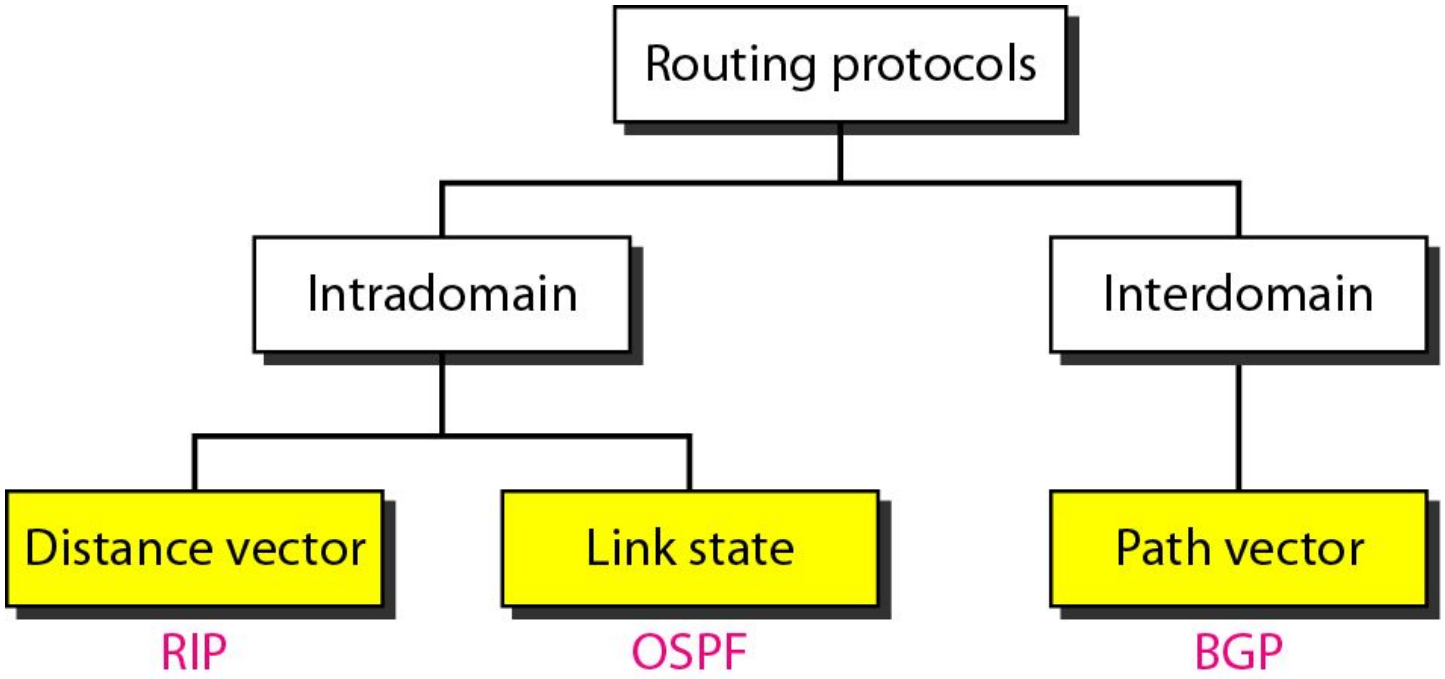
- A routing table can be either static or dynamic. A static table is one with manual entries. A dynamic table is one that is updated automatically when there is a change somewhere in the Internet. A routing protocol is a combination of rules and procedures that lets routers in the Internet inform each other of changes.

Autonomous systems

- Autonomous system (AS) or domain is a set of routers or networks administered by a single organisation.



Popular Routing Protocols

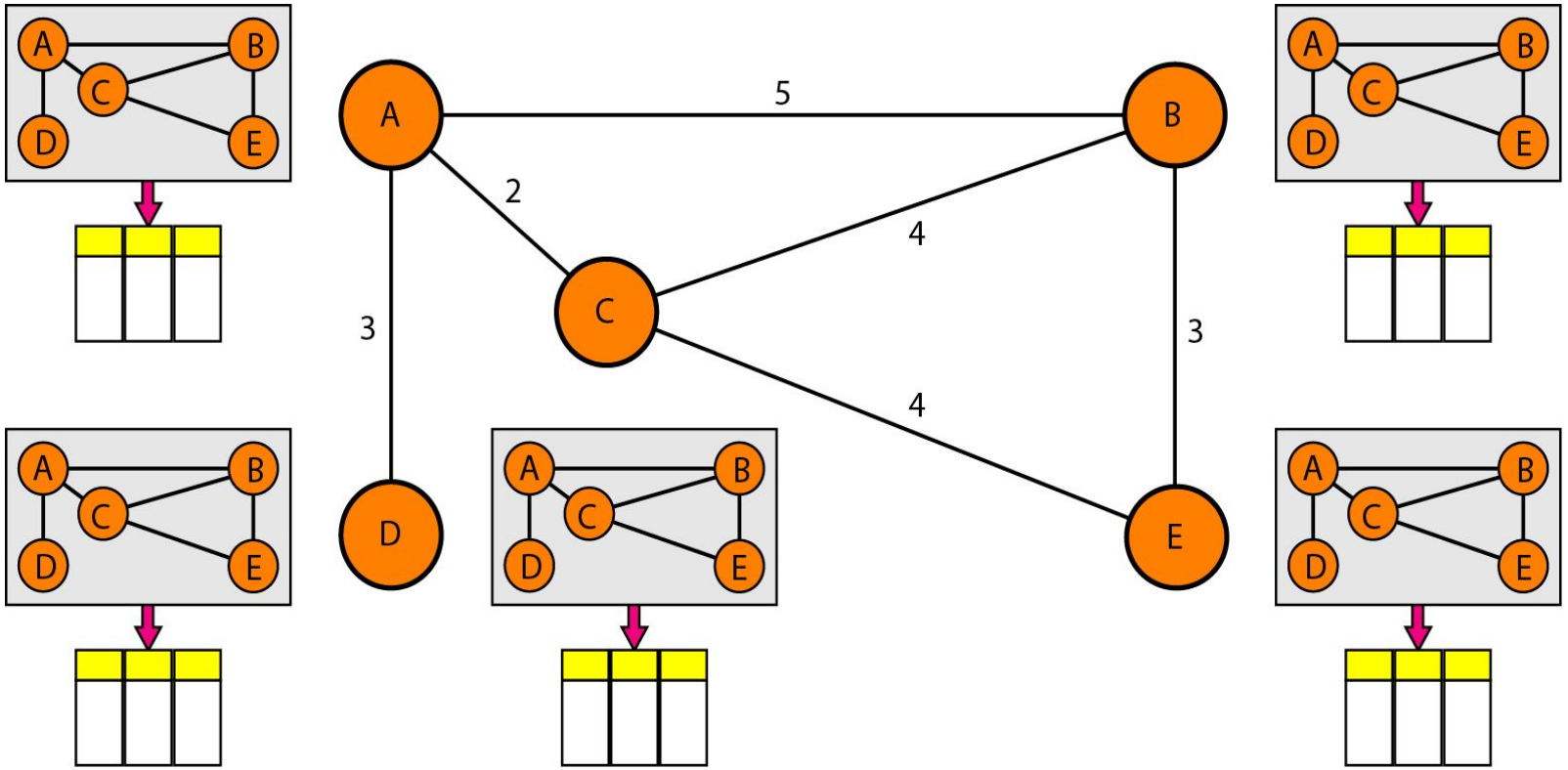


Link State Routing

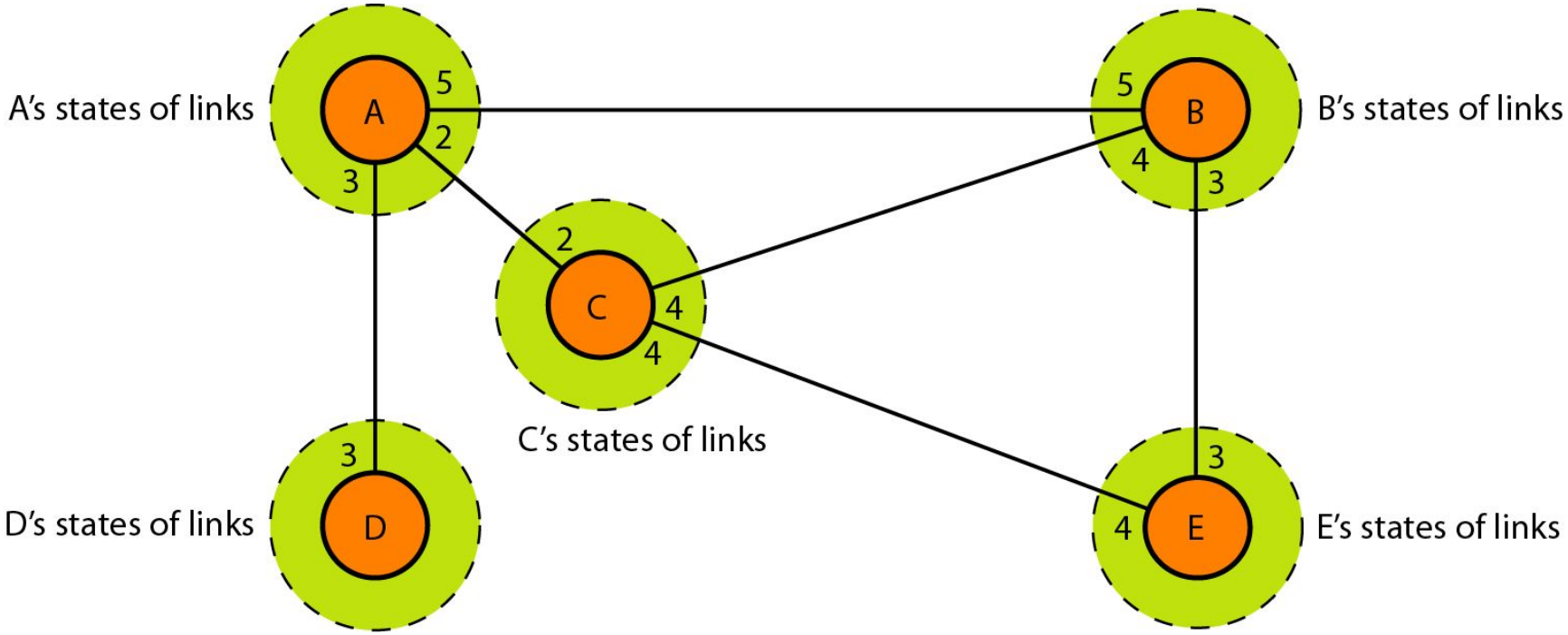
Link-State Routing

- Each node in the domain has the entire topology of the domain.
- The node can use Dijekstra's algorithm to build a routing table.
- Analogous to a city map.

Concept of link state routing



Link state knowledge



Link State

- Strategy
  - send to all nodes (not just neighbors)
  - information about directly connected links (not entire routing table)
- Link State Packet (LSP)
  - id of the node that created the LSP
  - cost of link to each directly connected neighbor
  - sequence number (SEQNO)
  - time-to-live (TTL) for this packet

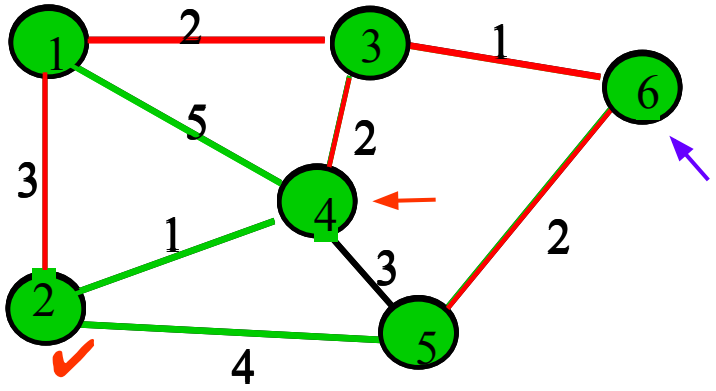
Link State

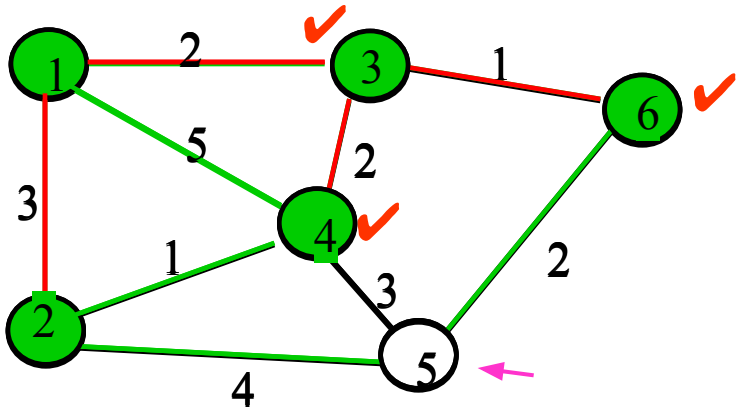
- Reliable flooding
  - store most recent LSP from each node
  - forward LSP to all nodes but one that sent it
  - generate new LSP periodically
    - increment SEQNO
  - start SEQNO at 0 when reboot
  - decrement TTL of each stored LSP
    - discard when TTL=0

Route Calculation

- Dijkstra's shortest path algorithm

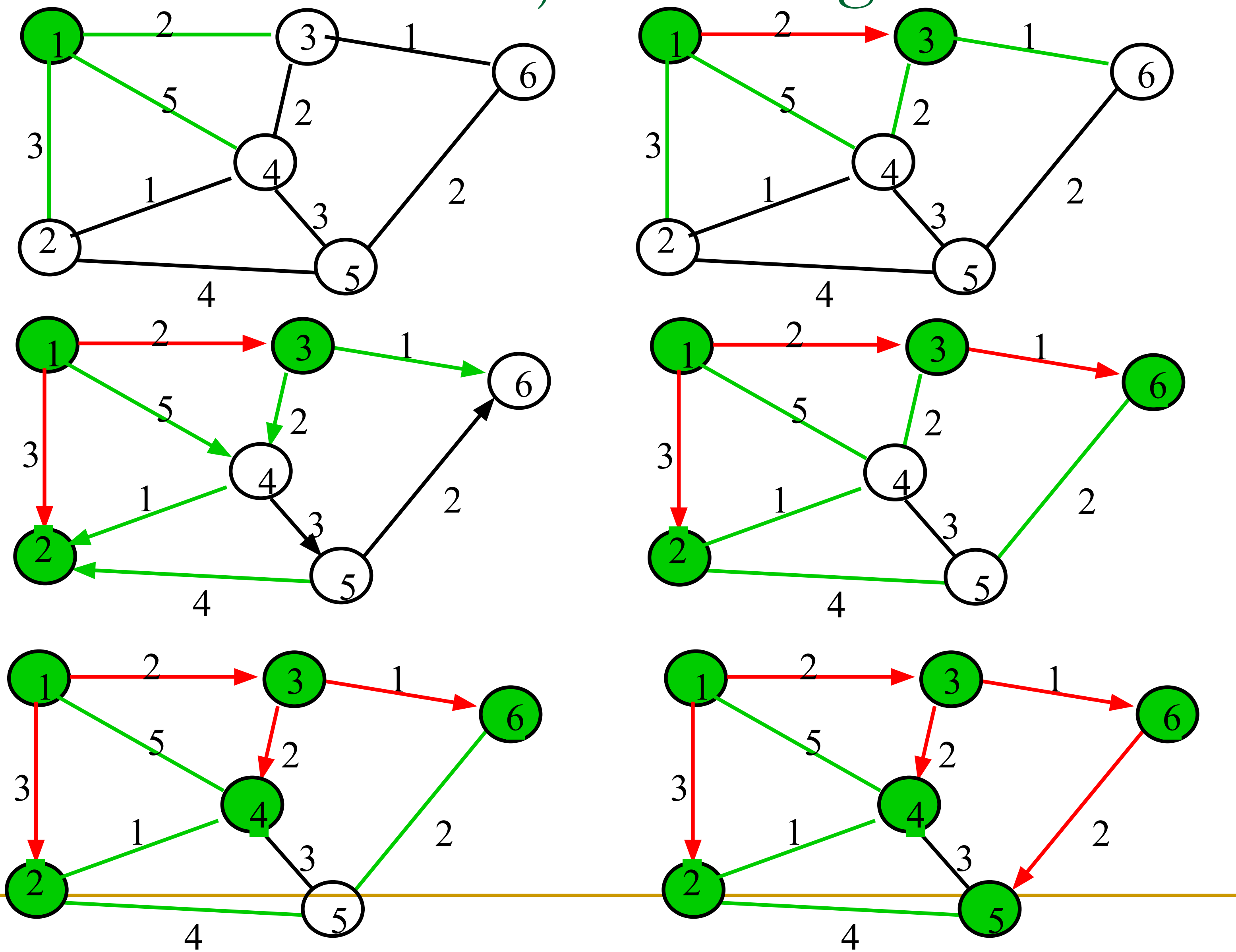
Execution of Dijkstra's algorithm





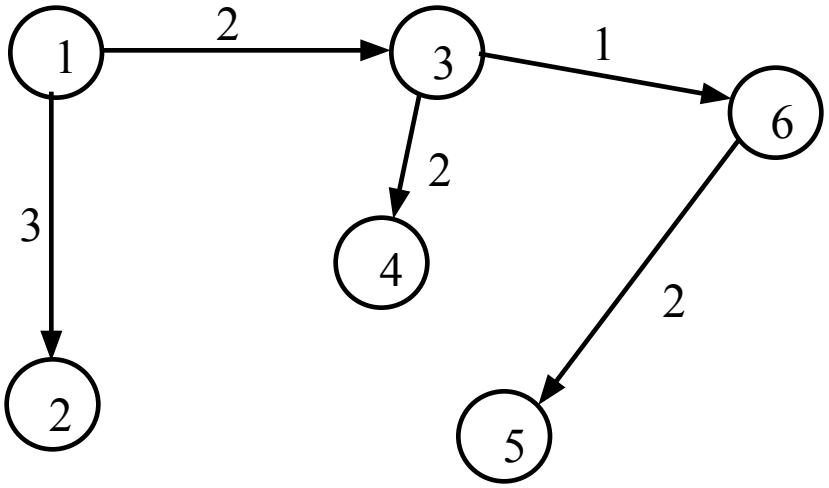
Iteration	N	D_2	D_3	D_4	D_5	D_6
Initial	{1}	3	2 ✔	5	∞	∞
1	{1,3}	3 ✔	2	4	∞	3
2	{1,2,3}	3	2	4	7	3 ✔
3	{1,2,3,6}	3	2	4 ✔	5	3
4	{1,2,3,4,6}	3	2	4	5 ✔	3
5	{1,2,3,4,5,6}	3	2	4	5	3

Shortest Paths in Dijkstra's Algorithm



Properties of Link State

- By keeping the predecessor node at every iteration, we can build the shortest path tree
- Loop free
- can use any cost metric, even multiple cost metric
  - one routing table for each metric
  - must use the same metric along the path, identified in the packet
- needs state of every link in the network



- potentially faster, because nodes exchange link state and perform computation locally as opposed to Distance Vector where the computation is distributed
- can compute equal cost multipath

Reaction to Failure

- If a link fails,
  - Router sets link distance to infinity & floods the network with an update packet
  - All routers immediately update their link database & recalculate their shortest paths
  - Recovery very quick
- But watch out for old update messages
  - Add time stamp or sequence # to each update message
  - Check whether each received update message is new
  - If new, add it to database and broadcast
  - If older, send update message on arriving link

Distance Vector Routing

Distance Vector

- Each node maintains a set of triples
  - **`(Destination, Cost, NextHop)`**
- Node knows cost to each neighbor
- Directly connected neighbors exchange updates
  - periodically (on the order of several seconds)
  - whenever table changes (called *triggered* update)
- Each update is a list of pairs:
  - **`(Destination, Cost)`**
- Update local table if receive a “better” route
  - smaller cost
  - came from next-hop
- Refresh existing routes; delete if they time out

Distance Vector

Routing Table

For each destination list:

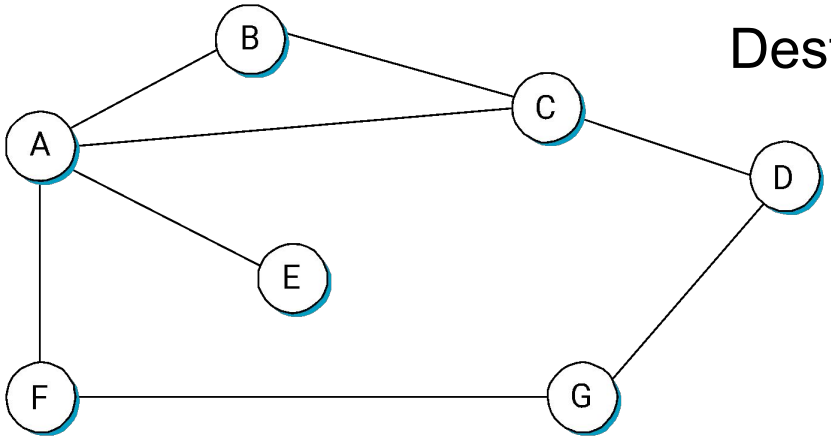
- Next Node
- Distance

dest	next	dist

Table Synthesis

- Neighbors exchange table entries
- Determine current best next hop
- Inform neighbors
  - Periodically
  - After changes

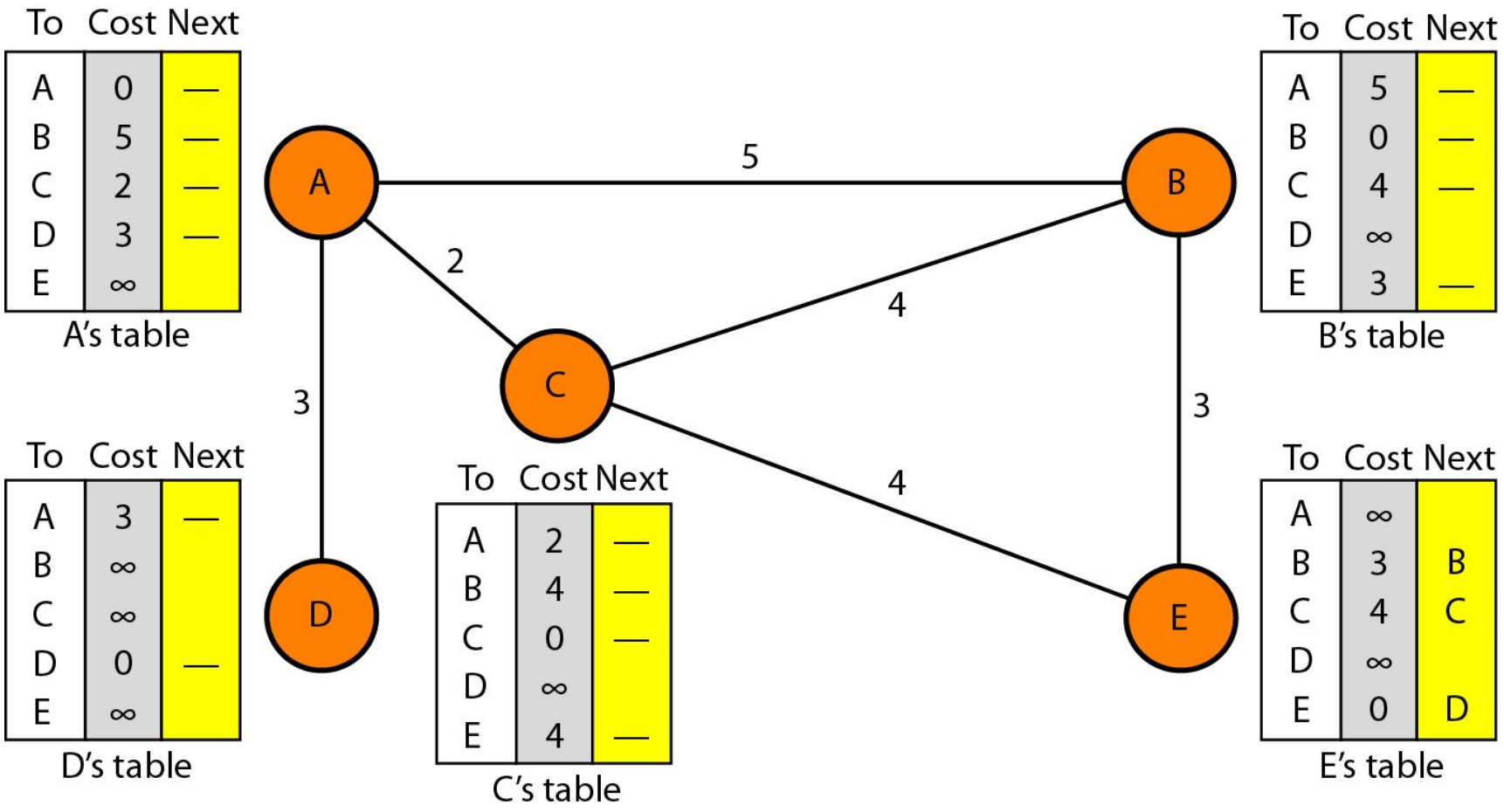
Example



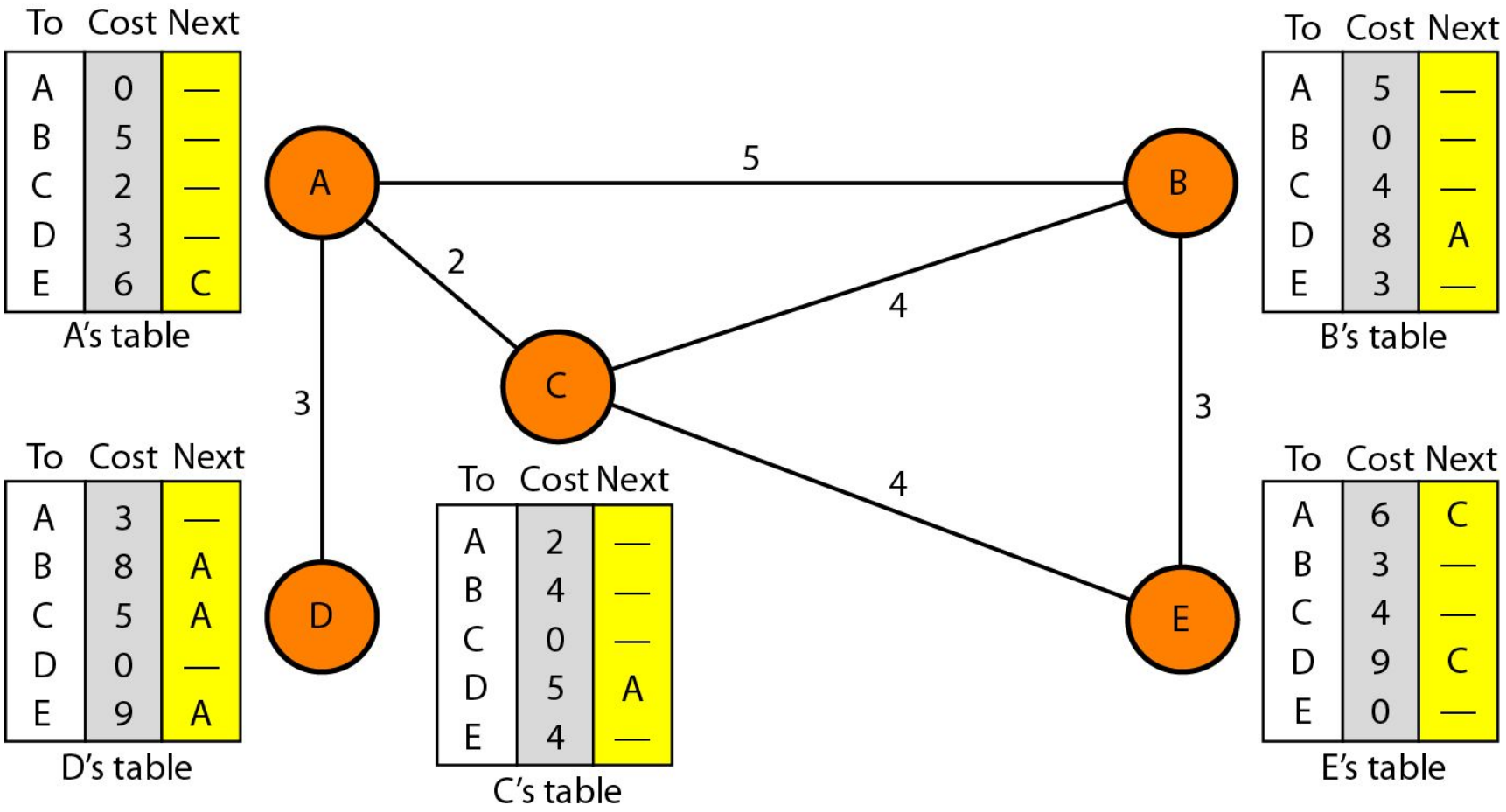
Node B

Destination	Cost	NextHop
A	1	A
C	1	C
D	2	C
E	2	A
F	2	A
G	3	A

Initialization of tables in Distance Vector Routing



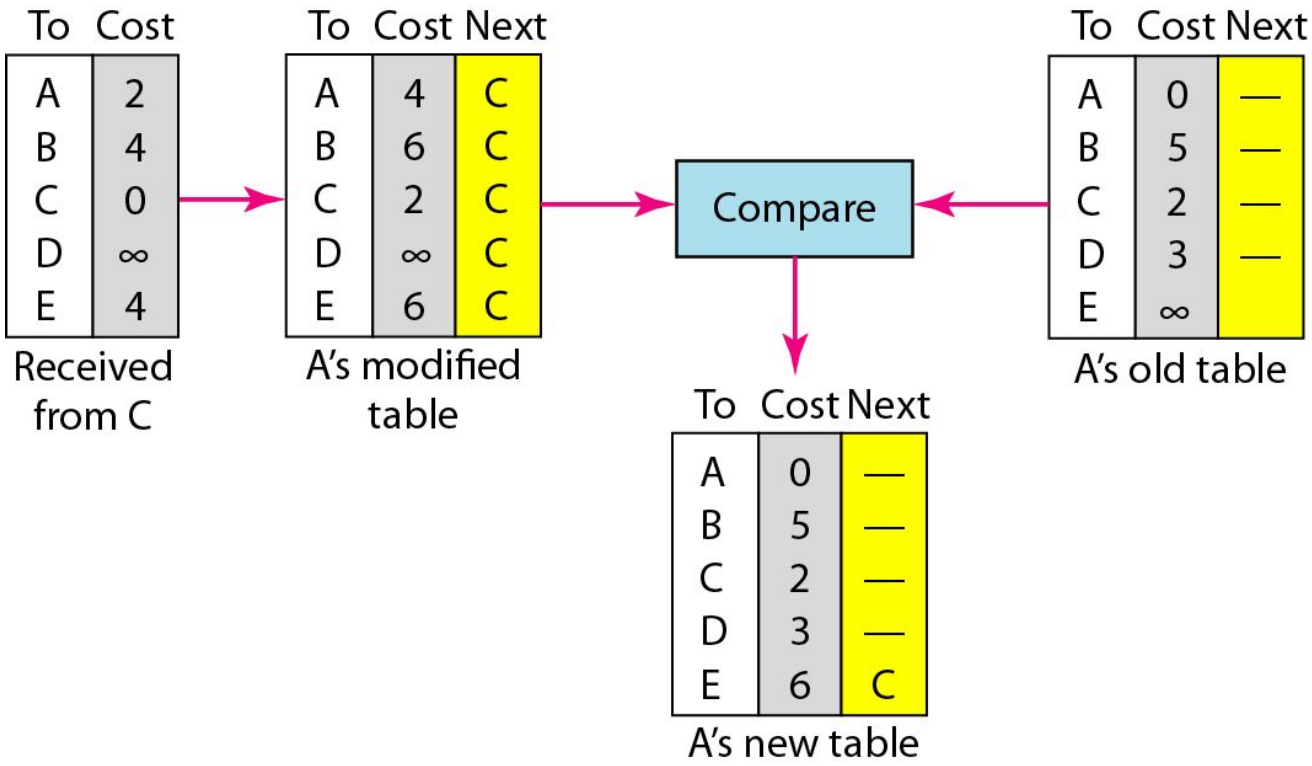
Distance Vector Routing Tables



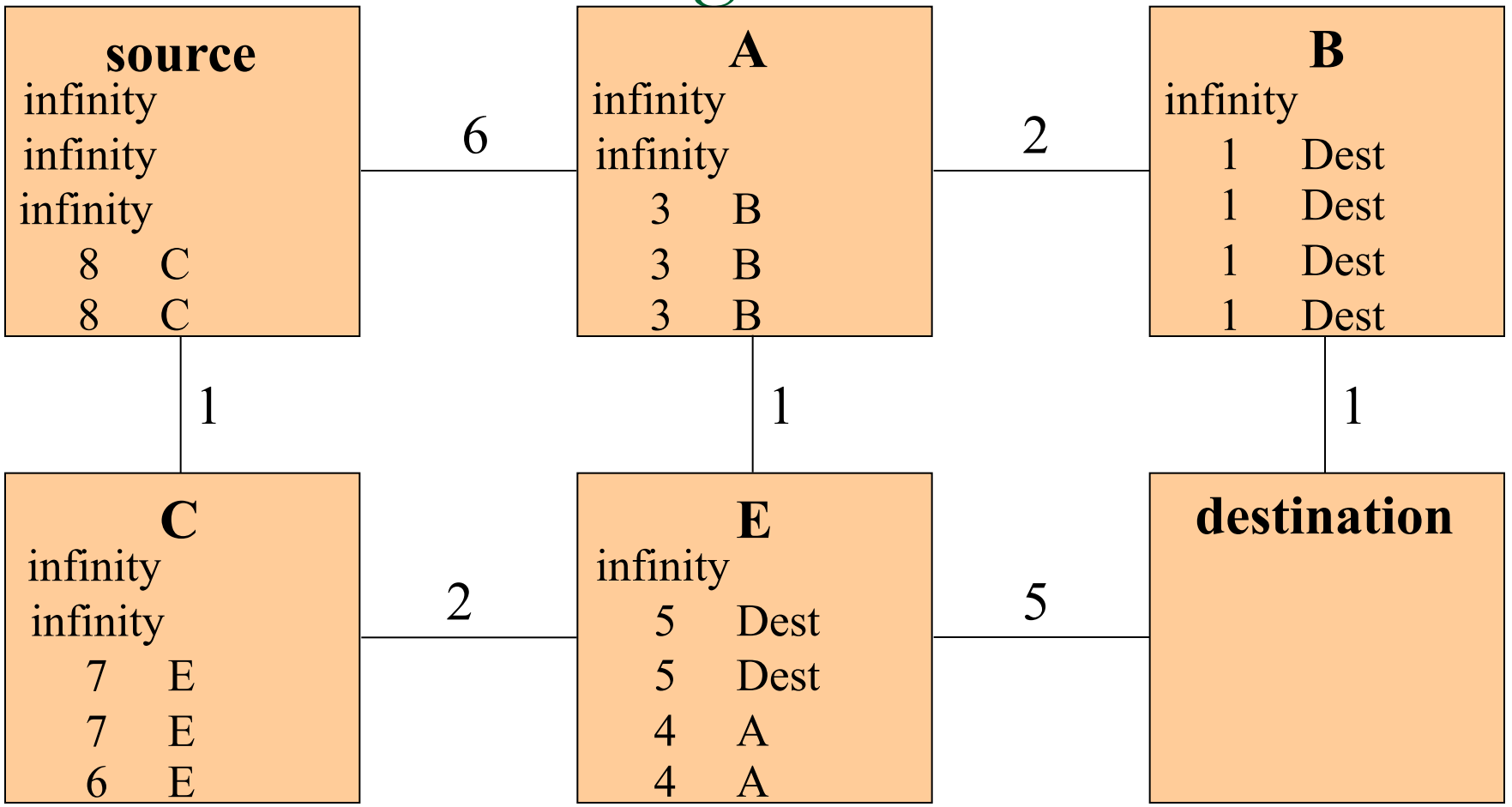
Note

In distance vector routing, each node shares its routing table with its immediate neighbors periodically and when there is a change.

Updating in Distance Vector Routing



Bellman-Ford Algorithm



- After *n* iterations, nodes at distance *n* hops along the shortest path have correct information